

Sindh Madressatul Islam University Karachi

Object Oriented Programming



SPRING 2026

Lecture 4 of 16
Engr. Majid Kaleem

WEEK 04 - AGENDA

- Abstract classes vs. Interfaces in Java

<https://onecompiler.com/java>

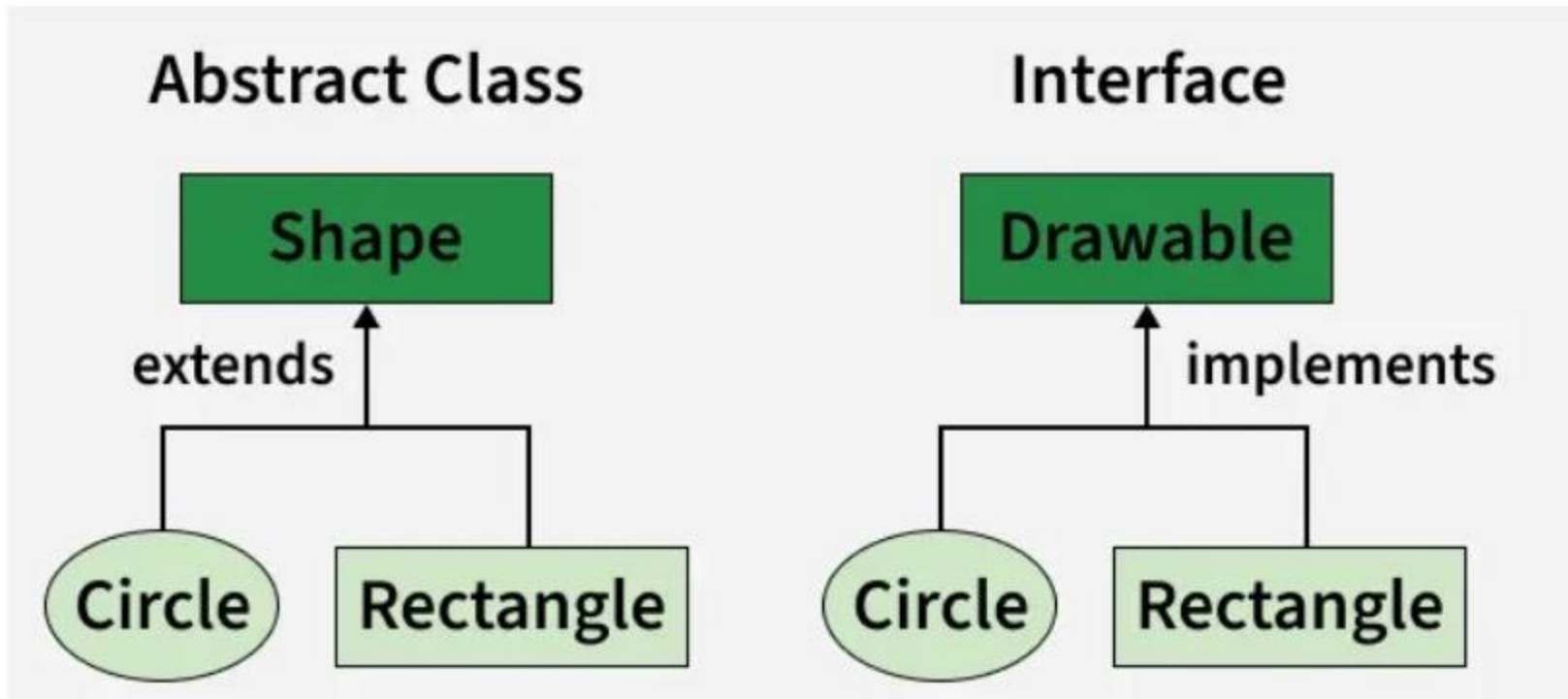
WEEKLY AGENDA

TENTATIVE WEEKLY DATES		TENTATIVE TOPICS
1	26 th -Jan – 30 th Jan	INTRO. TO OBJECT-ORIENTED PROGRAMMING, PROCEDURAL VS.OOP, OOP PRINCIPLES, ETC.
2	02 nd Feb – 06 th Feb	CLASSES & OBJECTS, JAVA NAMING CONVENTION, JAVA PACKAGES, & THE FOUR PILLARS OF OOP
3	09 th Feb – 13 th Feb	METHODS & MODULAR PROGRAMMING
4	16 th Feb – 20 th Feb	ABSTRACT CLASSES & INTERFACES ASSIGNMENT #1 & QUIZ #1
5	23 rd Feb – 27 th Feb	USE OF STATIC KEYWORD IN JAVA
6	02 nd Mar – 06 th Mar	POLYMORPHISM
7	09 th Mar – 13 th Mar	INTERFACES ASSIGNMENT #2 & QUIZ #2
8	16 th Mar – 20 th Mar	EXCEPTION HANDLING
← MID TERM EXAMINATIONS (23rd MAR 26 – 27th MAR 2026) →		
9	30 th Mar – 03 rd Apr	COLLECTIONS FRAMEWORK
10	06 th Apr – 10 th Apr	FILE HANDLING
11	13 th Apr – 17 th Apr	GUI PROGRAMMING
12	20 th Apr – 24 th Apr	APPLICATION DEVELOPMENT ASSIGNMENT #3 & QUIZ #3
13	27 th Apr – 01 st May	DESIGN PRINCIPLES & PATTERNS
14	04 th May – 08 th May	ADVANCED OOP CONCEPTS
15	11 th May – 15 th May	CASE STUDIES AND INDUSTRY PRACTICES ASSIGNMENT #4 & QUIZ #4
16	18 th May – 22 nd May	PRESENTATIONS & REVISION WEEK
← FINAL TERM EXAMINATIONS (25th MAY 26 – 29th MAY 2026) →		

Note

- This schedule is subject to revisions as conditions may warrant.*
- Topics will be covered in sequence no matter if the university observes any planned or unplanned holidays.*
- The information in this course outline is subject to revision as conditions may warrant.*

ABSTRACT CLASS & INTERFACE IN JAVA



Feature	Abstract Class	Interface
Instantiation	Cannot be instantiated	Cannot be instantiated
Methods	Can have both abstract and concrete methods	Methods are abstract by default (Java 8+ allows default and static methods)
Variables	Can have variables of any type (static, non-static, final, non-final)	Variables are public static final by default
Inheritance	A class can extend only one abstract class	A class can implement multiple interfaces
Constructors	Constructors are allowed	Constructors are not allowed
Access Modifiers	Members can have any access modifier (private, protected, public)	Methods are public by default

WHAT IS AN ABSTRACT CLASS?

- An abstract class in Java is a special type of class that **cannot** be instantiated directly and is meant to serve as a blueprint for other classes.
- It can contain:
 - **Abstract methods** (without a body) that must be implemented by **subclasses**.
 - **Concrete methods** (with a body) that can be reused by subclasses.
 - **Fields** (variables) and even constructors, unlike interfaces.
- In short, an abstract class lets you define common behavior and state while forcing **subclasses** to provide specific implementations for certain methods.

WHAT IS AN ABSTRACT CLASS?

- Abstract classes are used when you want to *share common code* among related classes but still enforce that each *subclass* must implement certain methods.

USES OF ABSTRACT CLASSES

- Provide a **base** class with both **abstract** (unimplemented) and **concrete** (implemented) methods.
- Allow partial abstraction (some methods defined, some left for subclasses).
- Useful when classes share state (fields) and behavior.
- Can include constructors to enforce initialization logic.
- Ideal when you want to provide a common template but allow **subclasses** to fill in details.

WHY SHOULD WE USE ABSTRACT CLASSES?

- **Reason:**
 - Abstract classes let you define a common base with shared state (fields) and behavior (methods), while leaving some methods unimplemented for **subclasses** to fill in.
- **Real-life example:**
 - Imagine a **BankAccount** abstract class. All accounts (Savings, Current) share fields like balance and methods like deposit(). But the way withdraw() works may differ, so you declare it abstract.
- **Code:**

WHY SHOULD WE USE ABSTRACT METHODS?

- **Reason:**
 - Abstract methods enforce that every subclass must provide its own implementation. They act like placeholders for behavior.
- **Real-life example:**
 - A **Vehicle** abstract class with an abstract method `move()`. Cars, bikes, and airplanes all move differently, so each subclass must implement `move()`.
- **Code:**

WHY SHOULD WE USE INTERFACES?

- **Reason:**
 - Interfaces define a contract that multiple unrelated classes can follow. They allow multiple inheritance of types and are great for designing APIs or capabilities.
- **Real-life example:**
 - A **Payable** interface. Both Employee and Invoice can be “payable,” even though they are unrelated classes.
- **Code:**

USES OF INTERFACES

- Define a contract (only method signatures, constants, and default/static methods).
- Achieve full abstraction (no state, unless using static/final fields).
- Support multiple inheritance (a class can implement multiple interfaces).
- Useful for plug-and-play design (different classes can implement the same interface differently).

WHEN TO USE?

- **Abstract class:** Use when you want a shared base with fields + methods and some methods left abstract.
- **Abstract method:** Use when you want **subclasses** to implement specific behavior.
- **Interface:** Use when you want to define a **contract** that can be applied across **unrelated classes**, supporting multiple inheritance.

To be continued...

```
If(anyQuestions)
{
    askNow();
}
else
{
    thankYou();
    submitAttendance();
    endClass();
}
```